

Step 1: Use a 64-bit OS and update it

Use Raspberry Pi OS (64-bit), preferably Lite, to save RAM. Then run:

```
bash
sudo apt update
sudo apt upgrade -y
sudo reboot
```

Check that you're on 64-bit after reboot (the output should say `aarch64`):

```
bash
uname -m
```

Step 2: Install Ollama

```
bash
curl -fsSL https://ollama.com/install.sh | sh
```

The article notes that the script sometimes crashes on the Pi. If that happens, use this version instead:

```
bash
curl -fsSL --http1.1 https://ollama.com/install.sh | sh
```

Verify the install:

```
bash
ollama --version
```

Ollama runs as a background service automatically. Ollama only uses the Pi's CPU, not its GPU, so expect modest speed.

Step 3: Pick a model for 8 GB RAM

The guide recommends models of about 4B parameters or less on a Pi 5. Good starting points:

Model	Command name	Notes
Gemma 3 1B	<code>gemma3:1b</code>	Fast, responsive
Qwen 0.5B	<code>qwen:0.5b</code>	Very fast, less accurate
Gemma 3 4B	<code>gemma3:4b</code>	Better answers, slower

The author tried 7B models on a Pi 5 8GB and found them accurate but too slow. I'd start with `gemma3:1b`, then move up to `gemma3:4b` if you want better quality.

Step 4: Download the model

```
bash
ollama pull gemma3:1b
```

List what's installed:

```
bash
ollama list
```



Step 5: Chat in the terminal

```
bash
ollama run gemma3:1b
```

You'll get a `>>>` prompt. Type your question and press Enter. Useful commands inside the chat:

- `/bye` exits the chat
- `/clear` resets the conversation context
- `/?` shows all available commands

To get speed stats (tokens per second) after each answer:

```
bash
ollama run gemma3:1b --verbose
```

That's already a working chatbot. The next steps are optional.

Step 6 (optional): Add a browser chat interface with Open WebUI

This gives you a ChatGPT-style page you can open from your phone or PC. It runs in Docker.

Install Docker:

```
bash
sudo apt install docker.io -y
```



The article runs the bundled `:ollama` image, which includes a second copy of Ollama inside the container. Since you already installed Ollama natively in Step 2, this cleaner variant connects to it instead and avoids downloading models twice:

```
bash
sudo docker run -d --network=host \
  -v open-webui:/app/backend/data \
  -e OLLAMA_BASE_URL=http://127.0.0.1:11434 \
  -e PORT=3000 \
  --name open-webui --restart always \
  ghcr.io/open-webui/open-webui:main
```

Step 6: Open WebUI (browser chat interface) in detail

6.1 Install Docker

```
bash
sudo apt install docker.io -y
```

Make sure it starts now and on every boot:

```
bash
sudo systemctl enable --now docker
```

Check it works:

```
bash
sudo docker --version
sudo systemctl status docker
```

Press `q` to leave the status screen. You should see `active (running)`.

6.2 Make sure Ollama is running

Open WebUI needs Ollama's service up first:

```
bash
curl http://localhost:11434
```

Expected reply: `Ollama is running`. If not, start it with `sudo systemctl start ollama`.

6.3 Start the Open WebUI container

```
bash
sudo docker run -d --network=host \
  -v open-webui:/app/backend/data \
  -e OLLAMA_BASE_URL=http://127.0.0.1:11434 \
  -e PORT=3000 \
  --name open-webui --restart always \
  ghcr.io/open-webui/open-webui:main
```

What each part does:

Part	Meaning
<code>-d</code>	Runs in the background
<code>--network=host</code>	The container shares the Pi's network, so it can reach Ollama at <code>127.0.0.1:11434</code>
<code>-v open-webui:/app/backend/data</code>	A Docker volume that keeps your accounts and chat history if the container is recreated
<code>-e OLLAMA_BASE_URL=...</code>	Tells WebUI where Ollama lives
<code>-e PORT=3000</code>	The web page will be on port 3000
<code>--restart always</code>	Starts automatically after a reboot
<code>--name open-webui</code>	Container name, used for the management commands below

Press Ctrl + C. You'll get your normal prompt back (smfahadpi5@SMFahadPi5:~ \$).

```
sudo docker ps -a
```

1. Read the logs

```
bash
sudo docker logs --tail 40 open-webui
```

What to do

1. Wait a few minutes, then check whether it has finished:

```
bash
sudo docker logs --tail 15 open-webui
```

You're looking for a line saying Unicorn is running, or an Open WebUI banner (large ASCII text). Once you see that, it's ready.

2. Check that the port is now listening:

```
bash
sudo ss -tlnp | grep -E '3000|8080'
```

If you see `:3000`, open `http://localhost:3000` in Chromium. If you see `:8080`, use `http://localhost:8080` instead.

```

smfahadpi5@SMFahadPi5:~$ sudo ss -tlnp | grep -E '3000|8080'
LISTEN 0      2048          0.0.0.0:3000      0.0.0.0:*        users:(("python3",pid=4114,fd=30))
smfahadpi5@SMFahadPi5:~$ sudo docker logs --tail 30 open-webui
[sudo] password for smfahadpi5:
2026-09-30 07:11:05.040 | INFO      | uvicorn.protocols.http.httptools_impl:send:485 - 127.0.0.1:47084 - "GET /api/v1/auths/ HTTP/1.1" 200
2026-09-30 07:11:05.062 | INFO      | uvicorn.protocols.http.httptools_impl:send:485 - 127.0.0.1:47084 - "GET /api/models HTTP/1.1" 200
2026-09-30 07:11:21.901 | INFO      | uvicorn.protocols.http.httptools_impl:send:485 - 127.0.0.1:45276 - "GET /api/v1/tools/ HTTP/1.1" 200
2026-09-30 07:11:21.913 | INFO      | uvicorn.protocols.http.httptools_impl:send:485 - 127.0.0.1:45276 - "GET /api/v1/skills/ HTTP/1.1" 200
2026-09-30 07:11:21.925 | INFO      | uvicorn.protocols.http.httptools_impl:send:485 - 127.0.0.1:45276 - "GET /api/v1/models/base/tags HTTP/1.1" 200

```

The first run downloads a large image (a few GB), so it can take several minutes. Watch progress with:

```

bash

sudo docker logs -f open-webui

```

Press **Ctrl+C** to stop watching. The container keeps running. WebUI is ready when the logs stop scrolling and show it listening.

6.4 Open it in a browser

Get your Pi's IP:

```

bash

hostname -I

```

On the Pi itself, go to `http://localhost:3000`. From another device on the same network, go to `http://<pi-ip>:3000`, for example `http://192.168.1.100:3000`. Use `http`, not `https`.

Startup can take 1 to 2 minutes after the container launches, so refresh if it doesn't load at first.

3. Open the page

In Chromium, go to `http://localhost:3000` and click **Reload**. Then follow Step 6.5:

1. Click **Get Started**.
2. Create the admin account (name, email, password).
3. Pick your model (`gemma3:1b`) from the dropdown at the top.
4. Type a message and press Enter.

2. Create your account

1. Click **Get Started**.
2. Enter a name, email and password, then click **Create Admin Account**. The email doesn't need to be real, since the account is stored only on your Pi.

3. Start chatting

1. Click the model dropdown at the top of the screen.
2. Select `gemma3:1b` (or whichever model you pulled).
3. Type a message and press Enter.

The first reply takes longer because Ollama loads the model into RAM. After that it's faster.

Fix 1: Turn off native function calling (most likely cause)

1. In the chat, click the **controls icon** in the top right (the sliders icon).
2. Open **Advanced Params**.
3. Find **Function Calling** and set it to **Default** (not **Native**).
4. Send your message again.

To make this permanent for the model:

1. Click your profile icon (bottom left) → **Admin Panel** → **Settings** → **Models**.
2. Click the pencil/edit icon next to `gemma3:1b`.
3. Open **Advanced Params**, set **Function Calling** to **Default**, and click **Save**.

Capabilities

- ☒ Vision
- ☒ Web Search
- ☐ Terminal
- ☐ Status Updates

- ☒ File Upload
- ☐ Image Generation
- ☐ Usage
- ☐ Memory

- ☒ File Context
- ☒ Code Interpreter
- ☐ Citations
- ☒ Builtin Tools

Default Features

- ☐ Web Search
- ☐ Code Interpreter

1. Untick these in Capabilities

- **Builtin Tools** (the most important one). Once it's off, the long Builtin Tools list in your second screenshot should disappear or stop applying, so you don't need to untick each item.
- **Web Search**
- **Code Interpreter**
- **Vision**, because `gemma3:1b` is text-only

You can leave **File Upload** and **File Context** ticked. They only matter if you upload documents, and they don't cause the tools error.

Leave **Default Features** (Web Search, Code Interpreter) unticked, as they already are.

Sign in to Open WebUI

Email
fahads@fahad1@gmail.com

Password
smfahad015

Sign in

If Email and Password incorrect:

```
2026-09-30 07:15:34.747 | INFO | uvicorn.protocols.http.httptools_impl:send:485 - 127.0.0.1:51720 - "GET /api/v1/auths/ HTTP/1.1" 200
2026-09-30 07:15:51.261 | INFO | uvicorn.protocols.http.httptools_impl:send:485 - 127.0.0.1:38260 - "GET /_app/version.json HTTP/1.1" 200
2026-09-30 07:15:59.839 | INFO | uvicorn.protocols.http.httptools_impl:send:485 - 127.0.0.1:35792 - "GET /_app/version.json HTTP/1.1" 200
smfahadpi5@SMFahadPi5:~$ sudo docker exec open-webui python3 -c "import sqlite3; c=sqlite3.connect('/app/backend/data/webui.db'); print(c.execute('select email from auth').fetchall())"
[('fahadsmfahad1@gmail.com',)]
smfahadpi5@SMFahadPi5:~$ sudo docker exec open-webui python3 -c "import sqlite3, bcrypt; h=bcrypt.hashpw(b'smfahad015', bcrypt.gensalt()).decode(); c=sqlite3.connect('/app/backend/data/webui.db'); c.execute('update auth set password=? where email=?',(h,'fahadsmfahad1@gmail.com')); c.commit(); print('done')"
done
smfahadpi5@SMFahadPi5:~$
```

Close the terminal window

Type this and press Enter:

```
bash
exit
```

Or press **Ctrl + D**. Either one closes the terminal window (or ends the SSH session, if you connected that way). You can also click the **X** at the top right of the window.

Exit a running program (the terminal is stuck)

- **Ctrl + C** stops a running command, such as a stuck `docker logs -f open-webui`.
- **q** closes screens like `systemctl status` and `less`.
- **/bye** exits the Ollama chat if you're inside `ollama run gemma3:1b`.
- **Ctrl + D** also exits the Ollama chat.

Shut down or reboot the Pi

Only do this when you want to power it off:

```
bash
sudo shutdown now
```

Wait until the green light stops flashing before you unplug the power. To reboot instead:

```
bash
sudo reboot
```

If you're using Raspberry Pi Connect, shutting down will disconnect your session, and you'll need to power the Pi back on physically to reconnect.

Step 7 (optional): Query it from Python or curl

Ollama also exposes a local API on port 11434, which is useful if you want to build the chatbot into your own program:

```
bash

curl http://localhost:11434/api/chat -d '{
  "model": "gemma3:1b",
  "messages": [{"role": "user", "content": "Hello!"}],
  "stream": false
}'
```

Useful maintenance commands

bash

```
ollama ps          # show models currently loaded in RAM
ollama rm gemma3:1b # delete a model to free disk space
sudo systemctl status ollama # check the Ollama service
sudo docker ps      # check Open WebUI is running
vcgencmd measure_temp # check the Pi's temperature
```